TIT TECHNOCRATS

Estd. : 1999

NBA ACCREDITED CSE (AIML, AIDS, AI, DS, CY, CSE, EX, EC, CS, ME, IT

nirf 2021-22 Ranking

Celebrating 26 YEARS of Excellence

Only Group of MP & CG with NBA in 3 Engg. Colleges

UNBEATABLE PLACEMENTS

Technocrats Group Campus, BHEL, Bhopal - 462021 MP, India
Phone : +91-755-2751679 | Mobile : +91-9826374295, +91-9893141968
e-mail : placements@titbhopal.net | website : http://www.technocratsgroup.edu.in

Campus Specific Training

By: Mr. Gautam Singh

Technocrats Group of Institutions
Technocrats Group Campus, BHEL, Bhopal - 462021 MP, India
Phone : +91-755-2751679
e-mail: info@titbhopal.net | website: www.technocratsgroup.edu.in

1. Fibonacci Series-:

```
class Fibonacci {
    public static void main(String[] args) {
        int n = 10, first = 0, second = 1;
        System.out.print("Fibonacci Series: " + first + " " + second);

        for (int i = 2; i < n; i++) {
            int next = first + second;
            System.out.print(" " + next);
            first = second;
            second = next;
        }
    }
}
```

2. Palindrome Number

```
import java.util.Scanner;

public class PalindromeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt(), temp = num, rev = 0;

        while (temp != 0) {
            rev = rev * 10 + temp % 10;
            temp /= 10;
        }
    System.out.println(num == rev ? "Palindrome Number" : "Not a Palindrome");
    }
}
```

Feature	String	StringBuilder	StringBuffer
Introduction	Introduced in JDK 1.0	Introduced in JDK 1.5	Introduced in JDK 1.0
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread Safe	Not Thread Safe	Thread Safe
Memory Efficiency	High	Efficient	Less Efficient
Performance	High(No-Synchronization)	High(No-Synchronization)	Low(Due to Synchronization)
Usage	This is used when we want immutability.	This is used when Thread safety is not required.	This is used when Thread safety is required.

Example: String Immutability

```
public class ImmutableStringExample {
    public static void main(String[] args) {
        String str1 = "Hello";
        str1.concat(" World");  // This does not modify str1

        System.out.println(str1);  // Output: Hello

        // To change the string, we need to assign it to a new reference
        str1 = str1.concat(" World");
        System.out.println(str1);  // Output: Hello World
    }
}
```

Mutable Strings (StringBuilder)

```
class MutableStringExample {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World"); // Modifies the existing object

        System.out.println(sb); // Output: Hello World
    }
}
```

Example of a Non-Thread-Safe Class (`StringBuilder`)

`StringBuilder` is **not thread-safe** because multiple threads modifying the same object can lead to inconsistent results.

Example: Multiple Threads Modifying `StringBuilder`

```
class Task implements Runnable {
    StringBuilder sb = new StringBuilder("Hello");

    public void run() {
        for (int i = 0; i < 5; i++) {
            sb.append(" World"); // Multiple threads modifying the same object
            System.out.println(sb);
        }
    }
}
```

```
class NonThreadSafeExample {
    public static void main(String[] args) {
        Task task = new Task();

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}
```

2 `StringBuilder` (Not Thread-Safe Example)

```
class StringBuilderExample {
    static StringBuilder sb = new StringBuilder("Hello");

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sb.append("A");
                System.out.println(Thread.currentThread().getName() + " -> " + sb);
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sb.append("B");
                System.out.println(Thread.currentThread().getName() + " -> " + sb);
            }
        });

        t1.start();
        t2.start();
    }
}
```

Example of a Thread-Safe Class (`StringBuffer`)

`StringBuffer` is **thread-safe** because it uses **synchronized methods**, preventing multiple threads from modifying it at the same time.

Example: Using `StringBuffer` (Thread-Safe)

```
class SafeTask implements Runnable {
    StringBuffer sb = new StringBuffer("Hello");

    public synchronized void run() {
        for (int i = 0; i < 5; i++) {
            sb.append(" World");
            System.out.println(sb);
        }
    }
}
```

```
public class ThreadSafeExample {
    public static void main(String[] args) {
        SafeTask task = new SafeTask();

        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);

        t1.start();
        t2.start();
    }
}
```

2 `StringBuffer` (Thread-Safe Example)

```
class StringBufferExample {
    static StringBuffer sb = new StringBuffer("Hello");

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sb.append("A");
                System.out.println(Thread.currentThread().getName() + " -> " + sb);
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                sb.append("B");
                System.out.println(Thread.currentThread().getName() + " -> " + sb);
            }
        });

        t1.start();
        t2.start();
    }
}
```

3. Palindrome String

```
import java.util.Scanner;

public class PalindromeString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str = sc.nextLine();
       String rev = new StringBuilder(str).reverse().toString();
        System.out.println(str.equals(rev) ? "Palindrome String" : "Not a Palindrome");
    }
}
```

4. Palindrome String

```
import java.util.Scanner;
public class PalindromeCheck {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string: ");
      String str = sc.nextLine();
      String rev = "";
      // Reverse the string using a loop
      for (int i = str.length() - 1; i >= 0; i--) {
         rev += str.charAt(i);
      }

      // Check if the original string matches the reversed string
      if (str.equals(rev)) {
         System.out.println("Palindrome");
      } else {
         System.out.println("Not a Palindrome");
      }
   }
}
```

5. Frequency of Characters in a String

```
import java.util.HashMap;
import java.util.Scanner;

public class FrequencyCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        HashMap<Character, Integer> freq = new HashMap<>();
        for (char ch : str.toCharArray()) {
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);
        }

        System.out.println("Character Frequency: " + freq);
    }
}
```

6(i). Prime Number Check

```
import java.util.Scanner;

 class PrimeNumber {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int num = sc.nextInt(), flag = 1;

      if (num <= 1) flag = 0;
      for (int i = 2; i <=num/2; i++) {
         if (num % i == 0) {
            flag = 0;
            break;
         }
      }

      System.out.println(flag == 1 ? "Prime Number" : "Not a Prime Number");
   }
}
```

6 (ii). Prime Number Check

```
import java.util.Scanner;

 class PrimeNumber {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number: ");
      int num = sc.nextInt(), flag = 1;

      if (num <= 1) flag = 0;
      for (int i = 2; i <=Math.sqrt(num); i++) {
         if (num % i == 0) {
            flag = 0;
            break;
         }
      }

      System.out.println(flag == 1 ? "Prime Number" : "Not a Prime Number");
   }
}
```

7. Maximum & Minimum in an Array

```
import java.util.Arrays;

class MinMaxArray {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1, 6, 9, 3};
        int min = Arrays.stream(arr).min().getAsInt();
        int max = Arrays.stream(arr).max().getAsInt();

        System.out.println("Minimum: " + min);
        System.out.println("Maximum: " + max);
    }
}
```

8. Reverse a Number

```
import java.util.Scanner;

public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt(), rev = 0;

        while (num != 0) {
            rev = rev * 10 + num % 10;
            num /= 10;
        }
        System.out.println("Reversed Number: " + rev);
    }
}
```

9. Sum of Digits of a Number

```
import java.util.Scanner;

public class SumOfDigits {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");

        int num = sc.nextInt(), sum = 0;

        while (num != 0) {

            sum += num % 10;

            num /= 10;

        }

        System.out.println("Sum of Digits: " + sum);

    }

}
```

10. Armstrong Number Check

```
import java.util.Scanner;

public class ArmstrongNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
         int num = sc.nextInt(), temp = num, sum = 0;
          int digits = String.valueOf(num).length();

        while (temp != 0) {
            sum += Math.pow(temp % 10, digits);
            temp /= 10;
        }
        System.out.println(num == sum ? "Armstrong Number" : "Not an Armstrong Number");
    }
}
```

Program: Frequency of Digits in a Number

```
import java.util.HashMap;
import java.util.Scanner;

public class DigitFrequency {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        String num = sc.next();

        HashMap<Character, Integer> freq = new HashMap<>();

        for (char digit : num.toCharArray()) {
            freq.put(digit, freq.getOrDefault(digit, 0) + 1);
        }

        System.out.println("Digit Frequency: ");
        for (char digit : freq.keySet()) {
            System.out.println(digit + " → " + freq.get(digit));
        }
    }
}
```

Program: Find Maximum & Minimum in an Array (Using Loop)

```
import java.util.Scanner;

public class MinMaxArrayLoop {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the number of elements: ");
      int n = sc.nextInt();
      int[] arr = new int[n];

      System.out.println("Enter the array elements:");
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }

      int min = arr[0], max = arr[0];

      // Loop through the array to find min and max
      for (int i = 1; i < n; i++) {
         if (arr[i] < min) {
            min = arr[i];
         }
         if (arr[i] > max) {
            max = arr[i];
         }
      }

      System.out.println("Minimum value: " + min);
      System.out.println("Maximum value: " + max);
   }
}
```

Program: Find Second Highest Value in an Array

```
import java.util.Scanner;

public class SecondHighest {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter the number of elements: ");
      int n = sc.nextInt();
      int[] arr = new int[n];

      System.out.println("Enter the array elements:");
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }

      int firstMax = Integer.MIN_VALUE;
      int secondMax = Integer.MIN_VALUE;

      // Loop to find first and second max

       }
}
```

```
for (int i = 0; i < n; i++) {
        if (arr[i] > firstMax) {
            secondMax = firstMax;  // Update second max before first max changes
            firstMax = arr[i];
        } else if (arr[i] > secondMax && arr[i] != firstMax) {
            secondMax = arr[i];
        }
    }

  if (secondMax == Integer.MIN_VALUE) {
        System.out.println("No second highest value found.");
    } else {
        System.out.println("Second highest value: " + secondMax);
    }

)
)
```

```
# Sort character
import java.util.*;
import java.lang.*;
import java.io.*;
class Codechef
{
	public static void main (String[] args)
	{
		char [] str = {'p','a','j','l'};
		for(int i=0;i<str.length;i++){
		   for(int j = i+1;j<str.length;j++){
		      if(str[i]>str[j]){
		         char temp = str[i];
		         str[i]=str[j];
		         str[j]= temp;
		      }
		   }
		}

		for(int i=0;i<str.length;i++){
		   System.out.println(str[i]);
		}
	}
}
```